GROUP PROJECT REPORT

Smart Waste-Bin System

Predicting when a bin will be full, with multiple linear regression and a Streamlit web app

Revised version, November 2026, updated after the lecturer's feedback





Group members

No.	Member name	Registration number
1	**Ndongozi Chris Arsene**	**26RP00610**
2	HABUMUKIZA Bon Ange	26RP00889

Acknowledgement

I would like to thank my lecturer, **Irankunda Innocent**, sincerely for the appreciation he showed for my project report presentation, and for the careful and constructive comment he left on the project afterwards.

His comment made me look again at decisions I had accepted too quickly: what counts as a full bin, how the prediction target was built, and how the data was split for testing. I chose to revise the project in response to his comment rather than to defend the mistakes I made in the earlier version. The comment identified real weaknesses, and correcting them has made the system, the evaluation and this report more accurate and more honest.

The lecturer's comment read as follows:

> *"Justify why 46% represents a full bin and explain how time-until-full labels were calculated without using future information in model inputs. Check that interpolation and collection cycles do not leak information across the time split. Cite the original dataset directly and explain the mismatch between weight calculated from fill percentage and independently entered weight. Support the reported results with saved evaluation outputs."*

The table below summarises the revisions. The detailed explanation follows on the next pages.

Point raised	Revision made
Why 46 % was a full bin	Threshold was an artefact of the sensor scale. Replaced by a defined capacity (200 kg = 100 %) and a collection level of 80 % (160 kg).
Labels without future inputs	Label kept separate from inputs. All six inputs use only information available at prediction time.
Leakage across the time split	Chronological 80/20 split and time-series cross-validation. Boundary effects from interpolation, cycles and calibration are stated openly.
Dataset citation and weight mismatch	Original dataset named and linked in Section 8. The weight is now the operator's input and the fill level is calculated from it. The simulation noise is explained.
Saved evaluation outputs	All results are read from metrics.json, evaluation.png and eda.png, and shown in the app.

Response to the Lecturer's Feedback

This section answers each part of the comment in turn. Where the original work was weak, it says so, and where a limitation remains, it is stated rather than hidden.

1. Why 46 % represented a full bin, and what replaced it

In the first version, the collection level was set automatically at 90 % of the typical weekly peak of the raw sensor reading. That peak was only about 51 % on the sensor's scale, so the threshold came out at 46 %. The figure was a product of the data, not a physical measure of how full the bin was, and calling it a full bin was not justified.

The revised version separates two ideas. The first is **capacity**: the sensor reading that the bin normally reaches in a weekly cycle (54.4 %, the 95th percentile of the cycle peaks) is treated as 100 % of capacity, and a capacity of 200 kg is attached to it. The second is the **collection level**: the bin is treated as ready for collection at 80 % of capacity (160 kg), which leaves a safety margin for waste added before the truck arrives. On the raw sensor scale this is about 43.5 %. The 80 % value is an operational design choice, not a property of the dataset, and it can be changed in the notebook (FULL_THRESHOLD). The app now shows the fill level, the collection level, the level remaining and the free space (Figures 4 to 6).

2. How the time-until-full labels were built without future information in the inputs

The label (hours_to_full) is the only quantity that looks forward in time, and it has to, because it is the answer the model learns to predict. For each reading, it is the time until the first reading of the same collection cycle that reaches the collection level. It is calculated afterwards from the historical record and never enters the model as an input.

The six inputs describe only what is known at the moment of prediction:

- **Fill level and weight:** the current reading.
- **Hours since collection:** the current time minus the start of the current cycle.
- **Average fill rate:** the fill level divided by the hours since collection.
- **Hour of day and weekend flag:** taken from the calendar.

Rows already at or above the collection level, and rows within the first hour after a collection, were removed, so the label is always positive and no division by near-zero hours occurs.

One limitation is stated openly. The start of each cycle is found from a centred 5-point median of the fill series, so locating a pickup uses up to two readings after it. This look-ahead only fixes where a cycle begins. No later fill value is given to the model. In live use the operator enters the real pickup time, so the input keeps the same meaning.

3. Interpolation, collection cycles and the time split

The evaluation uses a chronological split: the oldest 80 % of readings train the model and the newest 20 % test it, with no shuffling. The scaler used by the Ridge model is fitted on the training part only, and a five-fold time-series cross-validation is run on the training part as an extra check. The final model, refitted on all the data, is the one saved for the app. It is not used to produce the test scores. Three places where information could still cross the cut were reviewed in the notebook code:

- **Interpolation.** Gaps are filled only when they are at most 6 hourly readings long and lie between two valid readings. Linear interpolation uses the next valid reading, so a gap that straddles the cut could borrow one value from the other side. This affects at most six rows next to the boundary.
- **Collection cycles.** The label for a training row comes from the same cycle. A cycle that straddles the cut therefore has training labels that depend on a reading inside the test period. With a single cut, this is at most one cycle.
- **Capacity reference.** The 54.4 % reference is calculated from all cycle peaks before the split. It acts only as a fixed rescaling, but it is still a dataset-wide constant.

These effects are small, but they are not zero, so I do not describe the split as completely leak-free. The correction I recommend, listed under improvements, is to leave the boundary cycle out of training with a purge gap, and to calculate the capacity reference on the training period only. The reported scores should be read with that in mind.

4. Citing the original dataset, and the weight mismatch

Dataset. The readings come from the Kaggle dataset "Synthetic Smart City Infrastructure Sensor Data" (waste-sensor records, sensor IST-WASTE-003), which is synthetic and not field data. It is cited directly in Section 8. The Kaggle notebook by sumedh1507 is listed separately as secondary material that helped me understand the data. It does not replace the original dataset.

Weight. The dataset has no weight column, so the training weight was simulated as the fill level times 200 kg times a random factor with a mean of 1 and a spread of about 8 %. The training weight therefore differs slightly from the weight implied by the fill level. In the app the operator enters the weight and the fill level is calculated from it (weight divided by 200 kg), so the two are exactly consistent. Training data thus contains a small random mismatch that the app does not.

The effect on predictions is negligible. With coefficients of +0.026 per fill point and -0.011 per kilogram, an 8 % deviation at 40 kg (20 % fill) moves the estimate by well under 0.1 hour. More importantly, weight is simulated from the fill level, so it adds no independent information. The honest conclusion is that the weight input stands in for a future load-cell reading, and the accuracy reported here does not depend on it.

5. Saved evaluation outputs that support the results

The results in Section 5 are read from files saved by the training notebook, not typed in by hand:

Saved output	What it contains
artifacts/metrics.json	Data source, 13,342 rows, capacity and collection level, MAE, RMSE and R-squared for three models, coefficients and intercept
artifacts/evaluation.png	Actual versus predicted hours and residuals on the test set (Figure 2b)
artifacts/eda.png	Fill level over three weeks, hours until full, correlation map (Figure 2a)
artifacts/bin_model.pkl	The fitted model, its feature list and the collection level
artifacts/waste_bin.db	The cleaned readings shown in the Admin tab
Smart_Waste_Bin_Training.ipynb	The full pipeline and its printed output, including the time-series cross-validation scores

The Model and data tab of the app (Figure 6) displays the same values, so the report, the app and the saved file agree: the multiple linear regression reaches an MAE of 5.44 hours, an RMSE of 6.48 hours and an R-squared of 0.974, against 13.02 hours, 16.31 hours and 0.835 for the fill-only baseline. One gap remains. The cross-validation scores are printed in the notebook but not written to metrics.json, and saving them is on the improvement list.

Closing statement

I chose to revise the project instead of defending the earlier version. The comment showed me weaknesses I had not seen, and addressing them has produced a system that defines its terms clearly, evaluates itself more carefully, and documents its limits. I am grateful to Irankunda Innocent for the time and attention he gave the work.

1. The problem

Waste trucks usually follow a calendar, but bins do not fill on a calendar. Some overflow before the truck arrives, and others are emptied while still half empty. Every wasted trip costs fuel and time, and every overflowing bin is a health and cleanliness problem for the neighbourhood.

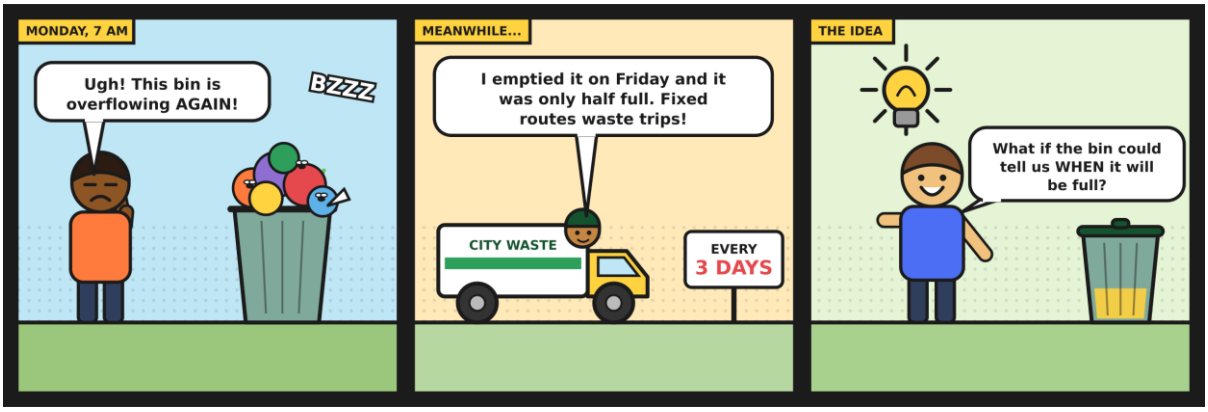


Figure 1. The problem in three panels.

Our idea: build a web app that answers one question, **when will this bin be full?** With that answer, collection can be planned around the bin instead of around the calendar.

2. Solution overview

The project follows four steps, from raw sensor data to a hosted application.

- **Collect and clean.** We started from the Kaggle smart-city sensor dataset and removed duplicates, spikes and gaps.
- **Train the model.** A multiple linear regression predicts the hours left until the bin is full.
- **Save the equation.** The intercept and coefficients are stored in a small file, metrics.json.
- **Serve it online.** A Streamlit app offers three tabs: Predict, Model and data, and Admin.

Behind the scenes, the Admin tab reads the cleaned readings from PostgreSQL, or from a read-only SQLite copy when no database is connected.

3. Data preparation

The dataset is a synthetic smart-city collection with one waste bin (sensor IST-WASTE-003). It has no weight column and no record of when the bin was emptied, so part of the work was building those pieces ourselves. The dataset is cited in full in Section 8.

Key figure	Value
Rows used for the model	13,342
Capacity and collection level	200 kg = 100 % of capacity; collection at 80 % (160 kg)
Sensor reading counted as 100 %	54.4 % (95th percentile of the weekly peaks)
Train / test split	Oldest 80 % / newest 20 % (split by time)

The cleaning steps were:

- Dropped duplicate timestamps and impossible values.
- Removed spikes more than 20 points from the local median.
- Filled gaps of up to 6 hours by interpolation, only between two valid readings.
- Detected each pickup as a fill drop of more than 20 points.
- Simulated weight from the fill level for a 200 kg bin, with about 8 % random variation, because the dataset has none.
- Rescaled the fill level so the bin's usual weekly peak (54.4 % on the raw sensor scale) counts as 100 % of capacity, then set the collection level at 80 % of capacity.

4. The model

The model is one equation: the predicted hours until full equal an intercept plus the sum of each coefficient times its input. The app reads these numbers from metrics.json and does the arithmetic itself, so no machine learning library is needed online.

Input	Coefficient	What it means
`intercept`	138.73	Starting point, in hours
`fill_pct`	+0.0256	Small; overlaps with weight
`weight_kg`	-0.0110	Small; weight is simulated
`hours_since_collection`	**-0.9980**	**Main driver: about -1 hour per hour**
`avg_fill_rate`	-0.0950	Faster filling, less time left
`hour`	+0.0323	Time of day, small effect
`is_weekend`	**+2.0373**	**Adds about 2 hours**

Table 1. Regression coefficients saved by the training notebook.

Worked example. With a weight of 40 kg (a 20 % fill level of the 200 kg bin) and about 24 hours since the last pickup, the equation returns about 115 h 23 min until the bin reaches the 80 % collection level (Figure 5).

Fill level and weight now carry almost the same information, because the weight is derived from the fill level. Their coefficients are small and have opposite signs, so neither should be read as a separate effect. The time since collection and the weekend flag do most of the work.

5. Results

We compared three models on the newest 20 % of the data: a baseline that uses only the fill level, the full multiple linear regression, and a Ridge regression. The values below come from the saved file metrics.json.

Model	MAE (h)	RMSE (h)	R-squared
Baseline (fill % only)	13.02	16.31	0.835
MLR (all features)	**5.44**	**6.48**	**0.974**
Ridge	5.44	6.48	0.974

Table 2. Test-set results.

Using all six inputs cuts the average error by about 58 %, from 13.0 hours to 5.4 hours. Ridge gives the same numbers, so plain multiple linear regression is enough for this data.

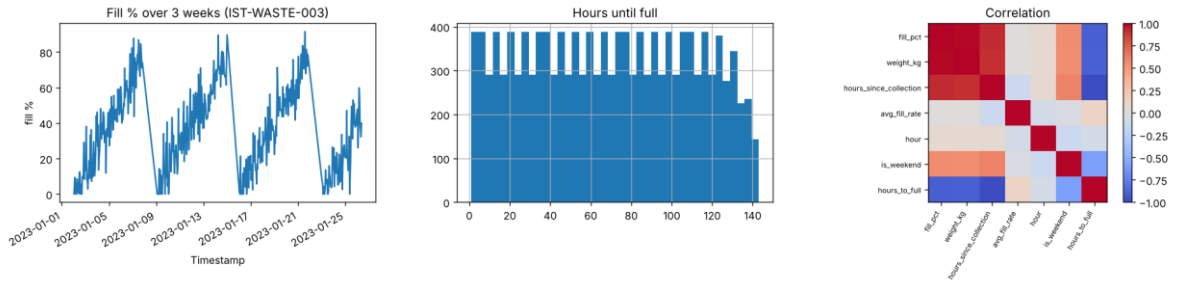


Figure 2a. Data overview. The bin fills and is emptied in weekly cycles, and hours until full spread evenly up to about 140 hours.

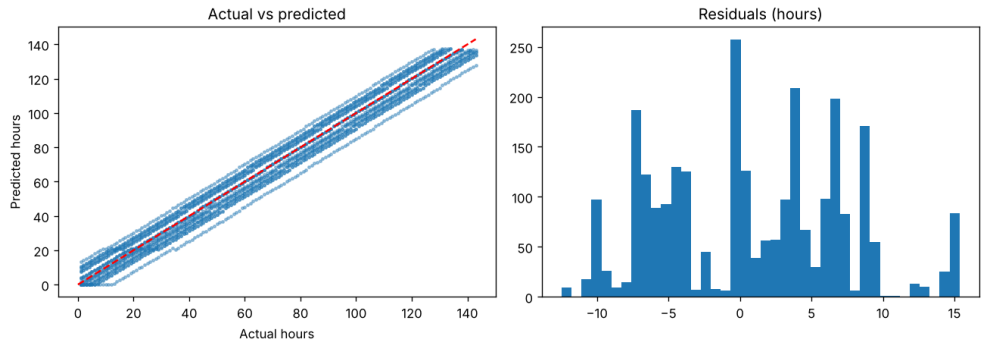


Figure 2b. Test-set fit. Predictions follow the red line, and errors stay within about 15 hours either way.

A caution. The data is synthetic, covers one bin, and weight is simulated from the fill level. The R-squared of 0.974 describes this dataset, not every real bin, and the small leakage risks listed in the lecturer-response section still apply.

6. How the app works

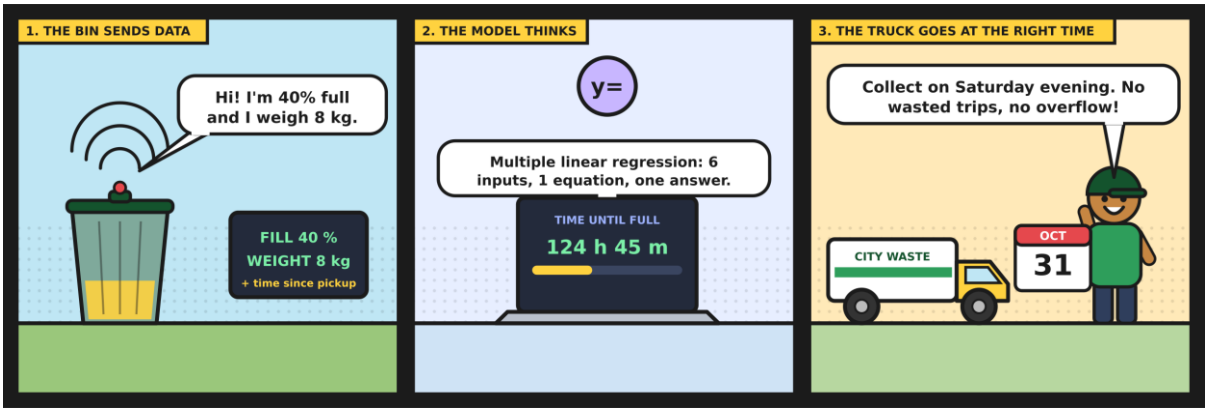


Figure 3. Inputs in, hours out.

6.1 Predict tab

The operator enters the current weight in the bin, in kilograms. The bin holds 200 kg when it is 100 % full, and the app turns the weight into a fill level automatically, so the two can never contradict each other. It then shows the collection level (80 %, or 160 kg), a progress bar, and the current collection: the date and time the waste level was recorded, with a tick box to use the current time as 'now'. Pressing **Predict remaining level** returns the level remaining before collection, the free space in the bin, a status message, and the model's estimate of the time left. Each prediction is added to a table for the session.

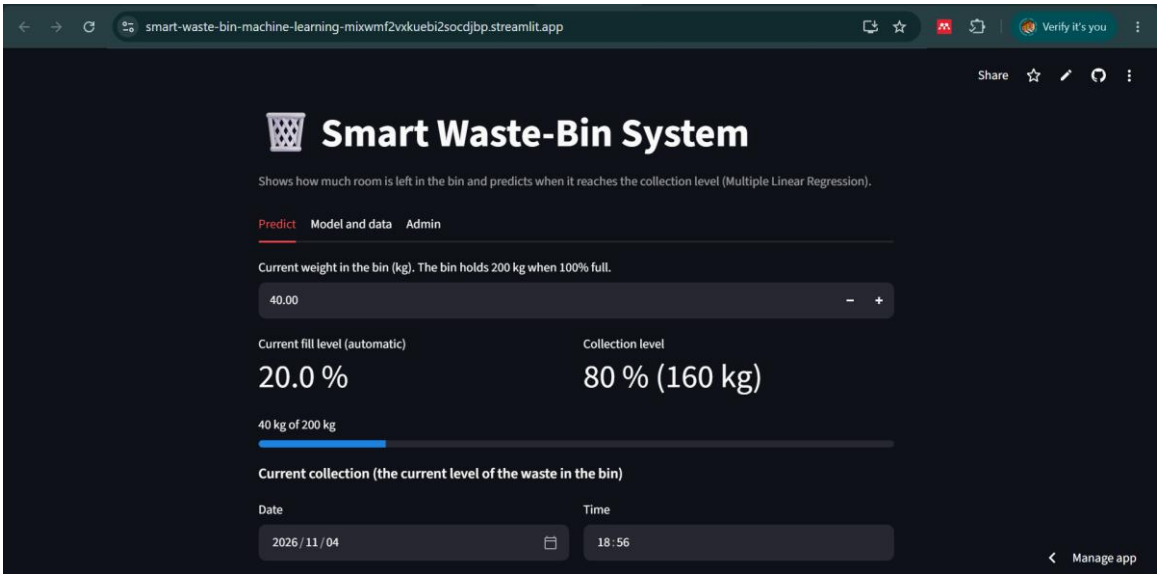


Figure 4. Predict tab: the weight is entered and the fill level is calculated automatically.

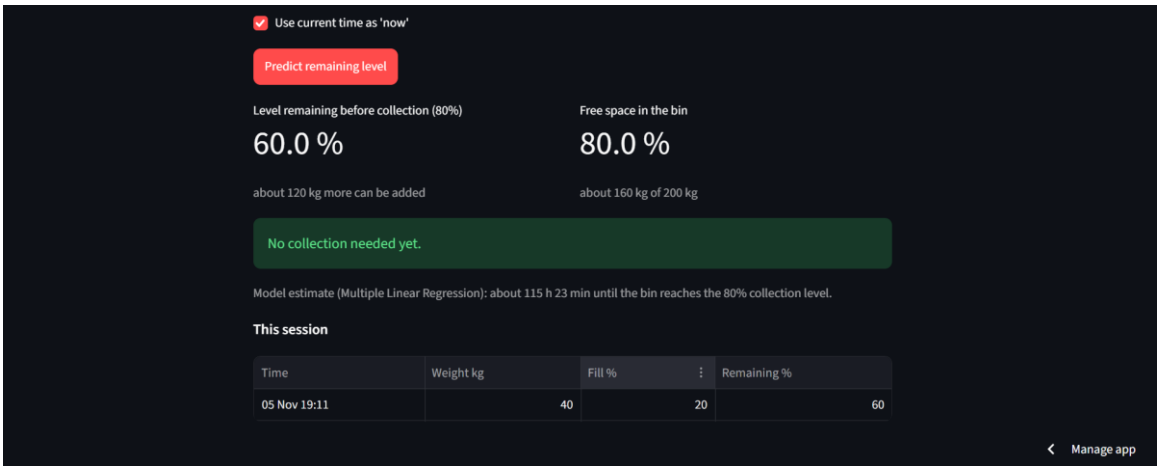


Figure 5. Result for 40 kg (20 % fill): 60 % remaining before collection, 80 % free space, and about 115 h 23 min until the 80 % level.

When the fill level reaches the 80 % collection level (160 kg), the app tells the operator to collect now. From 65 % upwards it shows a 'Collect soon' warning, and below that it reports that no collection is needed yet and gives the time estimate.

6.2 Model and data tab

This tab explains the answer. It shows the data source and row count, states openly that weight is simulated from the fill level, gives the bin capacity (200 kg = 100 %) and the 80 % collection level, explains how the sensor's 54.4 % weekly peak maps to 100 %, and compares the baseline, MLR and Ridge. A user can see how far to trust a prediction.

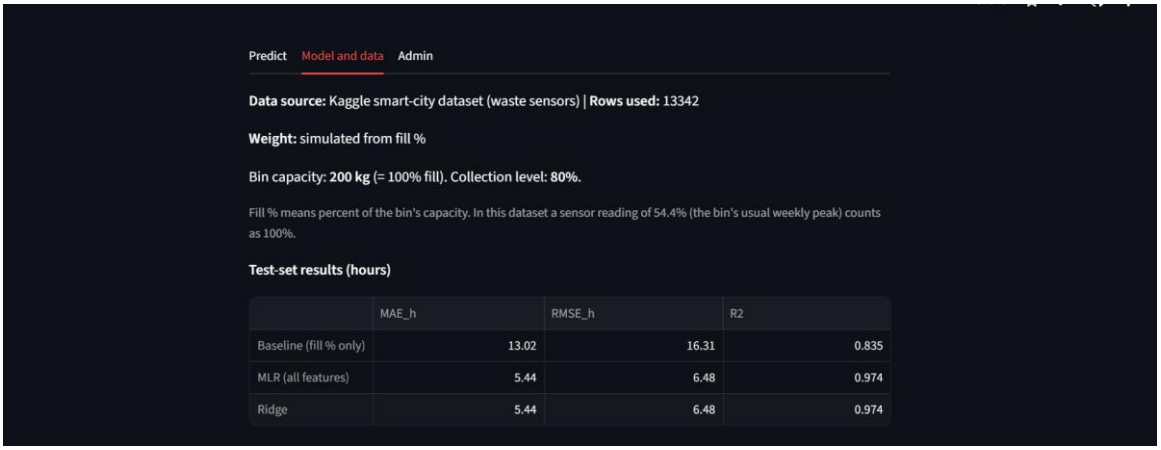


Figure 6. Model and data tab: data source, capacity, collection level and test-set results.

6.3 Admin tab

The Admin tab protects the stored readings behind a login. Passwords are stored as PBKDF2 hashes, and a session is locked after five failed attempts. With PostgreSQL connected, admins can also create viewer and admin users. When no database is reachable, as on the hosted demo, the tab falls back to a read-only SQLite copy of the data with a single admin login kept in Streamlit Secrets, and offers a CSV download.

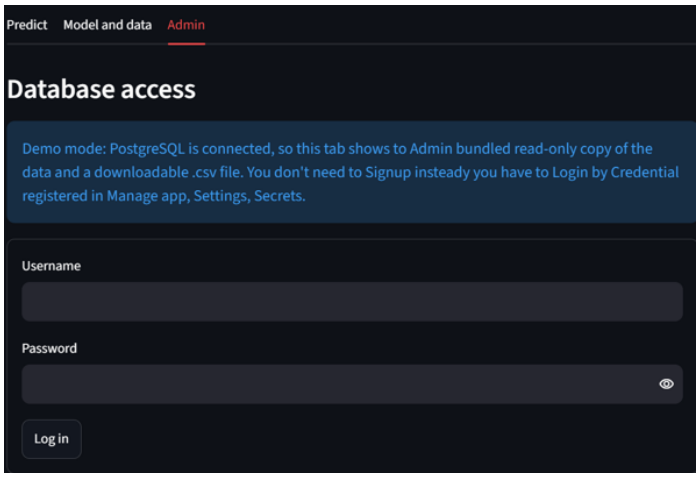


Figure 7. Admin login on the hosted app. The note explains that the login is the one registered in Streamlit Secrets.

7. Hosting

The app is deployed from GitHub on Streamlit Community Cloud.

- **Push to GitHub.** Code and artifacts go to the repository. The .env file stays out.
- **Streamlit builds it.** Community Cloud installs only pandas, streamlit and psycopg.
- **Secrets hold logins.** The admin user and password live in the app settings, never in the code.

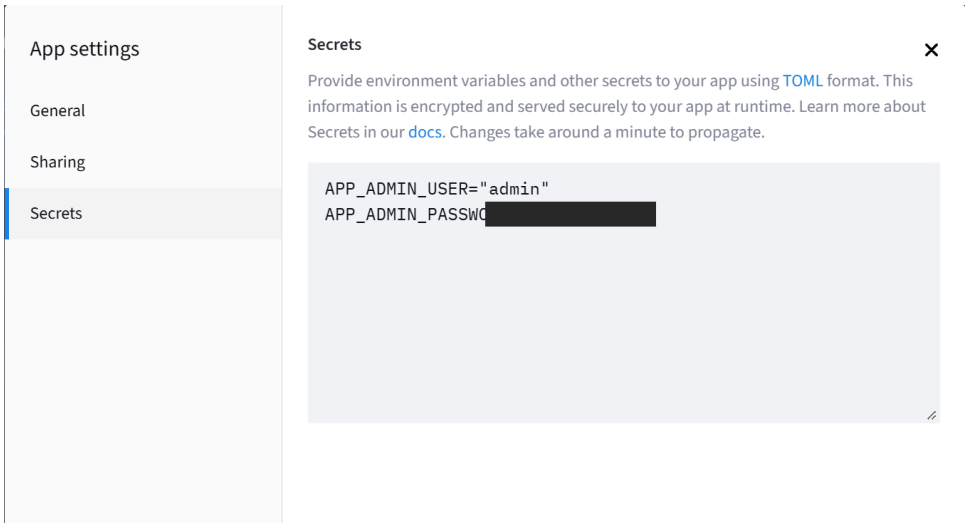


Figure 9. Streamlit Cloud app secrets. The password value is hidden in this copy of the screenshot.

Because predictions use the saved coefficients, the hosted app needs neither scikit-learn nor a model file.

A bug that taught us something

The Admin tab worked on a laptop but failed online. The PostgreSQL server we had installed only existed on that laptop, and the cloud could not reach it. The fix was a graceful fallback: the app checks what is available and uses PostgreSQL if it can, a read-only SQLite demo copy if it cannot, and a safe error message if a configured database is broken.

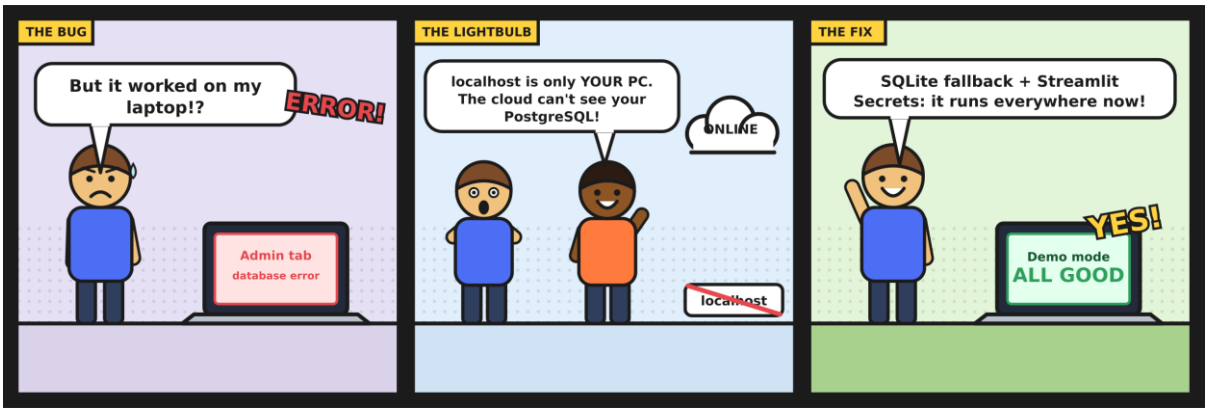


Figure 10. From "it worked on my laptop" to "it runs everywhere".

8. Project links

Item	Link
Hosted application	https://smart-waste-bin-machine-learning-mixwmf2vxkuebi2socdjbp.streamlit.app/
Source code (GitHub)	https://github.com/NdongoziChrisArsene/Smart-Waste-Bin-Machine-Learning
Original dataset	https://www.kaggle.com/code/sumedh1507/smart-city-sensor-data-insights

9. Individual reflection

Ndongozi Chris Arsene · 26RP00610

My contribution

My part of the project covered the whole path from raw data to a hosted app. I cleaned the Kaggle sensor file, which turned out to be messier than I expected, and then worked out how to turn a stream of fill percentages into something we could predict: the hours left until the bin is full. After that I trained the regression, checked it against a fill-only baseline and a Ridge model, and wrapped it in a Streamlit app with a login-protected Admin tab. Once the presentation was over, I went back through the work in response to the lecturer's feedback. I redefined what a full bin means, made the fill level calculate itself from the weight, retrained the model, redeployed the app and rebuilt this report around the new results.

What I learned

The model was the easy bit. Deciding what "full" means, and spotting when a pickup had happened, took far more thought than calling a fit function. I also learned to be suspicious of my own coefficients: fill level and weight turned out to be nearly the same signal, so their tiny coefficients say very little on their own. Splitting the data by time instead of at random showed me how easily a model can look better than it really is. And "it works on my machine" turned out to be a very different thing from "it is deployed".

Challenges I faced

Deployment was the hardest technical moment. The Admin tab ran perfectly on my laptop, then showed a database error online, because the PostgreSQL I had installed only existed on my machine. I rebuilt that part so the app checks what is reachable and falls back to a read-only demo copy, with the login kept in Streamlit Secrets so no password sits in GitHub. The data was the other hurdle: duplicates, gaps, spikes, no weight and no pickup record, so I had to simulate the weight and detect pickups myself. The hardest personal moment came later. The lecturer asked me to justify the 46 % threshold, and when I traced it back, I could not. Accepting that, instead of arguing for it, was uncomfortable but it was the right call. That also means the 0.974 R-squared describes this dataset, not every real bin.

What the revision taught me

Fixing the mistakes taught me more than the first build did. These are the skills and experience I gained from the update:

- **Thinking about data leakage.** I learned to trace every input and every label back to the moment of prediction and ask what was actually known then. I also learned that leakage hides in small places, such as interpolation across a split, a cycle that straddles the cut, and a constant calculated on the whole dataset.
- **Defining thresholds and units properly.** A full bin is a design decision, not something the data decides. Separating capacity (200 kg = 100 %) from the collection level (80 %) made the numbers explainable.
- **One source of truth for each value.** Calculating the fill level from the weight, rather than asking for both, removed a whole class of contradictory inputs.
- **Building a clearer Streamlit interface.** I reworked the Predict tab with a weight input, a metric row, a progress bar, date and time inputs, a 'now' tick box and a session table.
- **Reproducibility.** Saving metrics.json, the plots and the model means that every number in this report can be traced to a file, and I now check the report against those files.
- **Responding to critique professionally.** I learned to answer a comment with evidence and a fix, to state limitations plainly, and to keep the GitHub repository, the hosted app and the report in step with one another.

What I would improve

- Remove the boundary collection cycle from training (a purge gap), calculate the capacity reference on the training period only, and save the cross-validation scores in metrics.json.
- Train on many bins, and test a non-linear model against the linear one.
- Connect a live sensor (an ESP32 or Raspberry Pi), send SMS or email alerts, move to a cloud PostgreSQL database for full user management online, and add automated tests.